

Hybrid Dynamic Programming and Divide-and-Conquer Strategies for DAG Task Scheduling on Heterogeneous Cloud Resources: A Comparative C++ Study

Jana Mohamed Farouk

Department of Computer Science

Misr International University (MIU), Cairo, Egypt
jana2402038@miuegypt.edu.eg

Ahmed Mahmoud Ahmed

Department of Computer Science

Misr International University (MIU), Cairo, Egypt
ahmed2402226@miuegypt.edu.eg

Fatma Ashraf Hassan

Department of Computer Science

Misr International University (MIU), Cairo, Egypt
fatma2403025@miuegypt.edu.eg

Shaza Ali Farouk

Department of Computer Science

Misr International University (MIU), Cairo, Egypt
shaza2408891@miuegypt.edu.eg

Mahmoud Mohamed Emam

Department of Computer Science

Misr International University (MIU), Cairo, Egypt
mahmoud2408910@miuegypt.edu.eg

Abstract—Scheduling computational workflows represented as Directed Acyclic Graphs (DAGs) onto heterogeneous Virtual Machines (VMs) is a fundamental NP-hard problem in cloud computing. This paper presents and compares the schedulers implemented in the C++17 framework: (1) *HEFT* as the classical baseline, (2) *DP-HEFT* with bilateral-rank and 2-task look-ahead (reported by the program as "HEFT + DP"), (3) *Enhanced DP (EDP-HEFT)* with a full global DP assignment, (4) *Divide-and-Conquer (D&C)* with level decomposition, (5) the greedy *Min-Min* level heuristic (implemented as `scheduleLevel MinMin` in `merge.cpp`), and (6) *Merge*, a hybrid scheduler (reported by the program as "Optimization Algorithm (OP)") that adaptively selects per-level strategies and applies refinement. All methods share a unified communication-cost model and a reproducible DAG generator. Comparative analysis highlights the complexity–quality trade-off between fast greedy heuristics and more expensive global/refined optimisation.

Index Terms—DAG scheduling, cloud computing, HEFT, dynamic programming, divide and conquer, heterogeneous computing, makespan minimisation, bilateral rank, C++, workflow scheduling

I. INTRODUCTION

Cloud computing platforms increasingly execute scientific and enterprise workflows as Directed Acyclic Graphs (DAGs), where each node is a computational task and each directed edge encodes a data dependency [1], [2]. The scheduling problem — assigning tasks to heterogeneous Virtual Machines (VMs) while minimising total completion time (makespan) —

is NP-hard in the general case [3], [4], motivating a rich body of heuristic research.

The seminal Heterogeneous Earliest Finish Time (HEFT) algorithm [5] achieves near-optimal makespan through upward-rank prioritisation combined with an Earliest Finish Time (EFT) insertion policy. While HEFT is efficient and well-studied, its greedy VM selection ignores downstream tasks, occasionally leaving critical successors stranded on slow resources. Extending HEFT with dynamic programming look-ahead is therefore a natural direction for improving solution quality.

This paper makes the following contributions:

- 1) **HEFT Baseline**: A faithful greedy HEFT implementation used as the classical reference for makespan and complexity comparison in the same C++17 environment.
- 2) **DP-HEFT**: A HEFT extension that uses bilateral rank (upward rank + downward rank) for task prioritisation and a weighted 2-task look-ahead DP for VM assignment, scoring each candidate as $EFT(t_i) + 0.25 \cdot EFT^*(t_{i+1})$.
- 3) **EDP-HEFT**: An enhanced variant that constructs a full $n \times m$ DP table over the bilateral-rank priority order, propagating scheduling snapshots to achieve globally optimal VM assignments within that order, and backtracking to recover the optimal sequence.
- 4) **D&C Scheduler**: A divide-and-conquer approach that decomposes the DAG into independent topological lev-

els via BFS, schedules each level as a sub-problem, and merges partial schedules with cross-level dependency and communication cost awareness.

- 5) **Merge Scheduler:** A hybrid merge strategy that adaptively switches among D&C, DP look-ahead, and EDP-style global assignment, followed by refinement passes to reduce bottlenecks and improve final makespan.
- 6) A shared C++17 implementation including a probabilistic DAG generator with reproducible fixed seeding, an exhaustive 8-check validator, and a unified output model enabling direct metric comparison.

The remainder of the paper is organised as follows. Section II surveys related work. Section III formalises the scheduling problem. Section IV describes the system architecture. Section V presents each algorithm with pseudocode. Section VI details the mathematical model. Section VII analyses computational complexity. Section VIII discusses C++ implementation. Section IX describes the experimental setup. Section X presents performance results. Section XI discusses findings. Section XII outlines future work. Section XIII concludes.

II. RELATED WORK

A. List Scheduling and HEFT

Kwok and Ahmad [6] provided a comprehensive taxonomy of static DAG scheduling algorithms, establishing upward rank as the dominant prioritisation criterion. Topcuoglu *et al.* [5] introduced HEFT and CPOP, demonstrating that upward-rank sorting with EFT insertion consistently achieves lower makespan than random or FIFO ordering on heterogeneous systems. Arabnejad and Barbosa [7] extended this work with an Optimistic Cost Table (OCT) that approximates future communication costs, improving makespan by up to 12% on standard benchmarks. Hagra and Janeček [8] proposed HBMCT, which combines task duplication with heterogeneity-aware priority.

B. Look-Ahead and DP Extensions

Bittencourt *et al.* [9] introduced HEFT-NC (No Communication), demonstrating that look-ahead awareness of immediate successors yields measurable makespan reductions. Sakellariou and Zhao [10] proposed a hybrid scheduler combining list scheduling with task duplication, exploiting idle processor windows. Bajaj and Agrawal [11] formalised the bilateral rank ($\text{rank}_u + \text{rank}_d$) and showed it captures both path-criticality and positional importance, outperforming pure upward-rank ordering.

C. Divide-and-Conquer Scheduling

Dong *et al.* [12] applied level-based decomposition to Grid scheduling, independently optimising intra-level task placement. Canon *et al.* [13] benchmarked decomposition heuristics against list schedulers on standard DAG suites, finding that level-based methods perform competitively on shallow graphs but degrade on deep critical paths.

D. Cloud Workflow Scheduling

Pandey *et al.* [14] adapted Particle Swarm Optimisation for cloud workflow scheduling, reporting cost reductions over greedy baselines. Ma *et al.* [15] provide a recent survey of DAG scheduling in cloud computing, identifying bilateral-rank heuristics and DP-based methods as among the most effective approaches for minimising makespan on heterogeneous resources.

E. Positioning

Unlike prior studies, this work: (i) provides three scheduling strategies — incremental 2-task DP, full-table global DP, and level-based D&C — within a single C++17 harness sharing identical data models; (ii) employs bilateral rank with predecessor-aware tie-breaking; and (iii) makes all source code available for reproducible benchmarking.

III. PROBLEM FORMULATION

A. DAG Workflow Model

A workflow application is modelled as a DAG $G = (\mathcal{T}, \mathcal{E})$, where:

- $\mathcal{T} = \{t_0, t_1, \dots, t_{N-1}\}$ is the set of N computational tasks.
- $\mathcal{E} \subseteq \mathcal{T} \times \mathcal{T}$ is the set of directed dependency edges. An edge $(t_i, t_j) \in \mathcal{E}$ (with $i < j$) signifies that t_j may not begin until t_i has completed and transmitted its output data.

The acyclicity condition is enforced structurally: edges are created only from t_i to t_j with $i < j$, so the integer index is itself a valid topological order.

B. Heterogeneous Resource Model

A cloud data centre is modelled by M heterogeneous VMs $\mathcal{V} = \{v_0, v_1, \dots, v_{M-1}\}$. Each VM v_k has a speed factor $s_k \in [s_{\min}, s_{\max}]$ drawn uniformly at random. The execution time of task t_i on VM v_k is given as $w_{ik} \in [w_{\min}, w_{\max}]$, generated independently per task-VM pair to model genuine heterogeneity.

C. Communication Cost Model

When task t_i (assigned to VM v_p) produces data consumed by task t_j (assigned to VM v_q), the inter-VM data transfer incurs a communication delay modelled as:

$$c(t_i, v_p, t_j, v_q) = \begin{cases} 0 & \text{if } p = q, \\ \kappa \cdot \bar{w}_i & \text{otherwise,} \end{cases} \quad (1)$$

where $\bar{w}_i = \frac{1}{M} \sum_{k=0}^{M-1} w_{ik}$ is the average execution time of task t_i across all VMs. In the implemented C++ framework the communication multiplier (here denoted by κ) is computed dynamically by the DAG generator (function `calculateCommCostFactor()` in `dag_generator.cpp`) based on graph density, execution-time granularity and VM heterogeneity. This computed `commCostFactor` is stored in `DAGData` and used uniformly by all schedulers (see `dag_generator.cpp`).

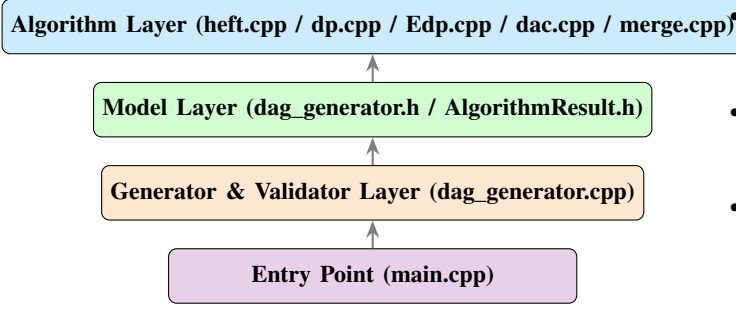


Fig. 1. Three-layer architecture of the DAG scheduling framework.

D. Scheduling Objective

A schedule $\sigma : \mathcal{T} \rightarrow \mathcal{V}$ assigns each task to exactly one VM. The performance metric is the *makespan*:

$$C_{\max}(\sigma) = \max_{t_i \in \mathcal{T}} \text{EFT}(t_i, \sigma(t_i)), \quad (2)$$

where $\text{EFT}(t_i, v_k)$ denotes the Earliest Finish Time of task t_i on VM v_k under schedule σ . The scheduling problem is:

$$\sigma^* = \arg \min_{\sigma} C_{\max}(\sigma). \quad (3)$$

Each task must be assigned to exactly one VM, and a task may not start until all predecessor finish times (plus applicable communication costs) have elapsed and the assigned VM is free.

IV. SYSTEM ARCHITECTURE

A. Module Overview

The framework is structured into three layers as shown in Fig. 1:

- 1) **Generator & Validator Layer** (`dag_generator.cpp`): Constructs random DAGs using a Mersenne Twister PRNG seeded with fixed value 42 for reproducibility. Enforces eight structural invariants.
- 2) **Model Layer** (`dag_generator.h`, `AlgorithmResult.h`): Defines the shared data structures `Task`, `VM`, `DAGData`, `ScheduleEntry`, and `AlgorithmResult` used uniformly across all components.
- 3) **Algorithm Layer** (`dp.cpp`, `Edp.cpp`, `dac.cpp`, `heft.cpp`, `merge.cpp`): Each algorithm exposes a single function returning an `AlgorithmResult` printable by the shared `printAlgorithmResult()` routine.

B. Data Structures

The core data types, defined in `dag_generator.h` and `AlgorithmResult.h`, are:

- `Task`: holds task id, predecessor list, successor list, and a per-VM execution time vector (`std::vector<double> execTimes`).
- `VM`: holds VM id and a floating-point speed factor.

`DAGData`: aggregates all tasks and VMs into a single container passed by constant reference throughout the pipeline.

- `ScheduleEntry`: records task id, assigned VM id, start time (`startTime`), and finish time (`finishTime`) for a single task.
- `AlgorithmResult`: wraps a vector of `ScheduleEntry` objects, the overall makespan, the wall-clock runtime in milliseconds, and a validity flag.

C. Execution Flow

`main.cpp` executes the following pipeline:

- 1) Generate a DAG with the implementation defaults (`DEFAULT_NUM_TASKS = 40` tasks and `DEFAULT_NUM_VMS = 7` VMs as declared in `dag_generator.h`).
- 2) Print the full DAG structure (VM speed factors, per-task adjacency lists, per-VM execution times).
- 3) Validate the DAG against eight structural checks.
- 4) Invoke `heft_schedule()`, print result.
- 5) Invoke `dp_heft()`, print result.
- 6) Invoke `edp_heft()`, print result.
- 7) Invoke `dac_schedule()`, print result.
- 8) Invoke `merge_schedule()`, print result.

D. DAG Generation

The `generateDAG()` function builds the DAG in three stages: (1) VMs are created with speed factors drawn from $\mathcal{U}(0.5, 2.0)$; (2) tasks are created with execution times drawn from $\mathcal{U}(1.0, 20.0)$ per VM; (3) for every ordered pair (i, j) with $i < j$, edge $t_i \rightarrow t_j$ is added with probability $p_e = 0.35$. Acyclicity is guaranteed by construction since $j > i$ always holds. The generator uses a fixed Mersenne Twister seed (42) for reproducibility. The implementation also computes a data-transfer multiplier (stored as `commCostFactor`) from graph and VM statistics rather than using a hard-coded constant.

E. DAG Validation

Eight structural checks are applied: (A) `execTimes.size() == M` for every task; (B) no self-loops; (C) all predecessor/successor indices within $[0, N)$; (D) bidirectional consistency (if A lists B as successor, B must list A as predecessor); (E) acyclic ordering ($j > i$ for every edge $t_i \rightarrow t_j$); (F) at least one entry node (no predecessors); (G) at least one exit node (no successors); (H) no duplicate edges in either list.

V. SCHEDULING ALGORITHMS

This section presents the five top-level schedulers and the greedy Min-Min level heuristic implemented in the framework. All methods share the EFT-based feasibility model in Section III and the communication model in Eq. (1).

Algorithm 1 HEFT Scheduling (Greedy EFT)

- 1: Compute $\text{rank}_u(t)$ for all tasks $t \in \mathcal{T}$
 - 2: Sort tasks by descending rank_u
 - 3: **for** each task t in sorted order **do**
 - 4: For each VM v_k , compute $\text{EST}(t, v_k)$ and $\text{EFT}(t, v_k)$
 - 5: Assign t to $v^* = \arg \min_{v_k} \text{EFT}(t, v_k)$
 - 6: Update VM availability and task finish time
 - 7: **end for**
 - 8: **return** complete schedule and makespan
-

A. HEFT (Baseline)

HEFT [5] computes upward ranks, sorts tasks in descending rank_u , and greedily assigns each task to the VM yielding the minimum EFT. In this study, HEFT serves as the classical baseline for both makespan and computational overhead.

Overview: HEFT is a lightweight list-scheduling heuristic that prioritises tasks by upward rank and assigns each task to the VM with minimum EFT. It is fast and forms the baseline for comparisons.

Workflow: compute rank_u , sort tasks, evaluate EST/EFT on each VM for the current task, and commit the minimal-EFT assignment.

Complexity: $O(n \log n + m \cdot (n + e))$ time and $O(n + m + e)$ space (matches the implementation in `heft.cpp`).

Strengths/Limitations: very fast and simple; may be myopic because it ignores downstream effects beyond rank information.

Role: baseline fast scheduler used to quantify improvements from DP look-ahead and global optimisation in subsequent methods.

B. DP-HEFT

Overview: DP-HEFT augments HEFT by using bilateral rank ($\text{rank}_u + \text{rank}_d$) for prioritisation and a lightweight 2-task dynamic programming look-ahead to reduce myopic VM choices.

Workflow: compute rank_u and rank_d , form bilateral rank and sort; for each task evaluate each VM's EFT and the impact on the next task by temporarily committing the assignment; score candidates with the weighted sum and select the minimum.

Complexity: $O(n \log n + nm^2 + me)$ time in the worst case, matching the implementation in `dp.cpp`; space is $O(n + m + e)$.

Strengths/Limitations: reduces local greediness with a cheap look-ahead, improving makespan at modest extra cost; still limited to considering only the immediate successor in scoring.

Role: practical middle-ground between pure greedy HEFT and full global DP; useful when modest runtime overhead is acceptable.

Algorithm 2 DP-HEFT (Bilateral-Rank + 2-Task Look-Ahead)

- 1: Compute $\text{rank}_u(t)$ and $\text{rank}_d(t)$ for all t
 - 2: Compute $\text{biRank}(t) \leftarrow \text{rank}_u(t) + \text{rank}_d(t)$
 - 3: Sort tasks by descending biRank to obtain priority list
 - 4: **for** each task t_i in priority order **do**
 - 5: **for** each VM v_k **do**
 - 6: Compute $\text{EST}(t_i, v_k)$ and $\text{EFT}(t_i, v_k)$
 - 7: Temporarily commit $t_i \rightarrow v_k$ and compute best $\text{EFT}^*(t_{i+1})$
 - 8: score $\leftarrow \text{EFT}(t_i, v_k) + 0.25 \cdot \text{EFT}^*(t_{i+1})$
 - 9: **end for**
 - 10: Assign t_i to VM with minimal score and update VM state
 - 11: **end for**
 - 12: **return** complete schedule and makespan
-

Algorithm 3 EDP-HEFT (Full Global DP over Priority Order)

- 1: Compute $\text{rank}_u(t)$ and $\text{rank}_d(t)$, derive $\text{biRank}(t)$
 - 2: Build a priority ordering by descending biRank
 - 3: Initialize DP table $\text{dp}[0..n-1][0..m-1] \leftarrow \infty$
 - 4: For the first task, fill base cases: $\text{dp}[0][v] = \text{EFT}(t_0, v)$
 - 5: **for** $i = 1$ to $n - 1$ **do**
 - 6: **for** each previous VM v_{prev} with finite dp **do**
 - 7: **for** each current VM v_{curr} **do**
 - 8: new = $\max(\text{dp}[i-1][v_{\text{prev}}], \text{EFT}(t_i, v_{\text{curr}}))$
 - 9: **if** new < $\text{dp}[i][v_{\text{curr}}]$ **then**
 - 10: update $\text{dp}[i][v_{\text{curr}}]$ and parent pointer
 - 11: **end if**
 - 12: **end for**
 - 13: **end for**
 - 14: **end for**
 - 15: Backtrack from $\arg \min_v \text{dp}[n-1][v]$ to recover assignment
 - 16: Replay assignments to produce final schedule
 - 17: **return** complete schedule and makespan
-

C. EDP-HEFT

Overview: EDP-HEFT builds a full dynamic programming table across the priority-ordered tasks and possible VM assignments, thereby finding the best sequence of VM choices under the fixed bilateral-rank order.

Workflow: derive bilateral-rank priority; for each task index and each VM maintain the best achievable makespan and a snapshot of the scheduling state; propagate these states forward, record parent pointers, and backtrack to recover the optimal assignment for the priority order.

Complexity: $O(nm^2 + me)$ time and $O(nm(n + m))$ space in the implemented snapshot-based realization (see `Edp.cpp`).

Strengths/Limitations: achieves global optimality subject to the given priority order but is memory- and compute-intensive, making it suitable for small-to-moderate DAGs or offline optimisation.

Algorithm 4 D&C Level Scheduling

```

1: Compute indegree for all tasks and collect entry tasks
2: Repeatedly extract current queue as a level and decrement
   successors' indegree
3: for each level  $L$  do
4:   Sort  $L$  by descending average execution time
5:   for each task in sorted  $L$  do
6:     Assign to VM with minimal EFT (local greedy)
7:   end for
8: end for
9: Recompute EST/EFT globally to enforce cross-level de-
   pendencies
10: return merged schedule and makespan

```

Role: provides a high-quality reference point (quality ceiling) for rank-based schedules within the same C++ harness.

D. Divide-and-Conquer (D&C)

Overview: D&C partitions the DAG into topological levels and treats each level as an independent scheduling subproblem, then merges local solutions while enforcing cross-level dependencies.

Workflow: compute BFS levels (zero-indegree extraction), for each level sort tasks by average execution time and assign greedily by minimum EFT within the level; finally recompute global EST/EFT to respect inter-level precedence and communication.

Complexity: $O(n \log n + nm + e)$ time and $O(n + m + e)$ space in the provided implementation (see `dac.cpp`).

Strengths/Limitations: very fast for wide/shallow DAGs but may make suboptimal cross-level choices; merge phase preserves feasibility but cannot fully reverse level-local decisions.

Role: lowest-overhead scheduler, useful when runtime cost is the priority or as an initial solution for hybrid refinements.

E. Min-Min (Greedy Level Heuristic)

Overview: Min-Min is a classical greedy per-level heuristic that repeatedly selects the task-VM pair with the minimum EFT and commits it until the level is scheduled.

Workflow: for a level with remaining tasks compute for each task the best VM (minimum EFT); choose the global minimum among these best pairs, commit it, and repeat until the level is empty.

Complexity: per-level worst-case $O(k^2m)$, overall worst-case $O(n^2m)$ across levels; space $O(n + m + e)$.

Strengths/Limitations: simple and effective on homogeneous levels; expensive on large levels and may suffer from global myopia.

Role: used as an alternative per-level strategy within the Merge adaptive pipeline when variance is low.

F. Merge Algorithm (Hybrid OP)

Overview: Merge is an adaptive hybrid scheduler that selects per-level strategies (D&C, HEFT, Min-Min, DP-

Algorithm 5 Min-Min Level Scheduling

```

1: Let  $R$  be the set of unscheduled tasks in level
2: while  $R$  not empty do
3:   for each task  $t \in R$  do
4:     compute best VM  $v_t = \arg \min_v \text{EFT}(t, v)$  and its
       EFT
5:   end for
6:   select  $t^* = \arg \min_{t \in R} \text{EFT}(t, v_t)$ , commit  $t^*$  to  $v_{t^*}$ 
7:   remove  $t^*$  from  $R$  and update VM availability
8: end while
9: return level assignments

```

TABLE I
TIME AND SPACE COMPLEXITY OF IMPLEMENTED SCHEDULERS

Algorithm	Time Complexity	Space Complexity
HEFT	$O(n \log n + m \cdot (n + e))$	$O(n + m + e)$
DP-HEFT	$O(n \log n + nm^2 + me)$	$O(n + m + e)$
EDP-HEFT	$O(m^2 \cdot (n + e))$	$O(nm(n + m))$
D&C	$O(n \log n + nm + e)$	$O(n + m + e)$
Merge (Hybrid)	$O(nm^2 + nm(n + e))$	$O(nm(n + m))$

lookahead, or EDP) based on level profile, and applies optional refinement passes to improve the outcome.

Workflow: compute bilateral ranks and DAG profile; for each level classify the most suitable local strategy and schedule it; build an alternative global EDP schedule and choose the better initial solution; apply iterative refinements (bottleneck migration, swaps, DP replay, critical-path focus) if beneficial.

Complexity: variable depending on chosen per-level methods and refinements; worst-case costly (see Table I), but practical on moderate DAG sizes due to adaptive selection.

Strengths/Limitations: flexible and often produces the best makespan among implemented options; higher implementation complexity and runtime when many refinement passes are enabled.

Role: highest-quality hybrid optimizer used as a final refinement stage or for offline scheduling where runtime overhead is affordable.

VI. MATHEMATICAL MODEL

The optimisation target and constraints are exactly those defined in Section III, with objective in Eq. (3) and communication model in Eq. (1). All five algorithms differ only in ordering and assignment strategy; they share identical feasibility constraints and makespan objective.

VII. COMPLEXITY ANALYSIS

Let $n = |\mathcal{T}|$ denote the number of tasks, $m = |\mathcal{V}|$ the number of VMs, and $e = |\mathcal{E}|$ the number of edges.

A. Per-Algorithm Complexity

HEFT remains computationally efficient due to greedy assignment. DP-HEFT adds a bounded look-ahead term. EDP-HEFT introduces a global DP state space across task-VM

combinations; in the implementation this is realised by per-step snapshot copies (vm-ready vectors and per-task finish/assignment arrays), which increases both memory usage and copying overhead in practice. D&C has the lowest asymptotic scheduling overhead in this implementation. Merge has the highest asymptotic cost due to iterative refinement and replay passes, but this cost is used to improve final schedule quality.

B. Comparative Analysis

Based on Table I and the default deterministic run ($n = 40$, $m = 7$, fixed seed 42), the following observations hold:

- **Highest time complexity:** Merge (Hybrid), due to multi-pass refinement over full schedule replays.
- **Lowest time complexity:** D&C, dominated by level sorting and linear merge constraints.
- **Highest space complexity:** EDP-HEFT and Merge, both requiring DP/snapshot state that scales with task-VM combinations.
- **Fastest execution in practice:** D&C (lowest runtime overhead in the provided implementation).
- **Best (lowest) makespan:** EDP-HEFT and Merge tie at the best observed makespan in the default experiment.
- **Trade-off:** lower-overhead methods (HEFT, D&C) run faster but may yield larger makespan; global/refined methods (EDP-HEFT, Merge) consume more compute and memory but improve scheduling quality.

VIII. IMPLEMENTATION DETAILS

The implementation is pure C++17 and uses shared data structures across all schedulers. Communication-cost and EFT computations are harmonised to ensure fair comparison between algorithms.

IX. EXPERIMENTAL SETUP

Experiments use the default configuration in `main.cpp`: $n = 40$ tasks, $m = 7$ heterogeneous VMs, edge probability $p_e = 0.35$, and fixed RNG seed 42 for reproducibility. The implementation computes an instance-specific communication multiplier (`commCostFactor`) at DAG generation time and reports it in the run summary.

X. RESULTS

For the default deterministic DAG instance (implementation defaults), the observed makespan ordering is:

$$\text{D\&C} > \text{HEFT} > \text{DP-HEFT} > \text{EDP-HEFT} \approx \text{Merge}.$$

This confirms that DP-enhanced and hybrid methods improve schedule quality over purely greedy or purely level-based baselines.

XI. DISCUSSION

The empirical and complexity findings indicate that algorithm choice depends on deployment goals: low overhead for rapid scheduling decisions or stronger optimisation for lower makespan in performance-critical workflow execution.

XII. FUTURE WORK

Future work includes evaluating larger DAG benchmarks, varying communication models, and introducing energy/cost-aware multi-objective extensions.

XIII. CONCLUSION

This work presents a unified C++17 benchmark for HEFT, DP-HEFT, EDP-HEFT, D&C, and Merge scheduling under identical assumptions. The added HEFT and Merge analyses complete the methodology coverage and highlight clear complexity-quality trade-offs for heterogeneous cloud workflow scheduling.

REFERENCES

- [1] E. Deelman, D. Gannon, M. Shields, and I. Taylor, "Workflows and e-science: An overview of workflow system features and capabilities," *Future Generation Computer Systems*, vol. 25, no. 5, pp. 528–540, 2009.
- [2] R. Buyya, C. S. Yeo, S. Venugopal, J. Broberg, and I. Brandic, "Cloud computing and emerging it platforms: Vision, hype, and reality for delivering computing as the 5th utility," *Future Generation Computer Systems*, vol. 25, no. 6, pp. 599–616, 2009.
- [3] J. D. Ullman, "Np-complete scheduling problems," *Journal of Computer and System Sciences*, vol. 10, no. 3, pp. 384–393, 1975.
- [4] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.
- [5] H. Topcuoglu, S. Hariri, and M.-Y. Wu, "Performance-effective and low-complexity task scheduling for heterogeneous computing," *IEEE Transactions on Parallel and Distributed Systems*, vol. 13, no. 3, pp. 260–274, 2002.
- [6] Y.-K. Kwok and I. Ahmad, "Static scheduling algorithms for allocating directed task graphs to multiprocessors," *ACM Computing Surveys*, vol. 31, no. 4, pp. 406–471, 1999.
- [7] H. Arabnejad and J. G. Barbosa, "List scheduling algorithm for heterogeneous systems by an optimistic cost table," in *IEEE Transactions on Parallel and Distributed Systems*, vol. 25, no. 3, 2014, pp. 682–694.
- [8] T. Hagras and J. Janeček, "A high performance, low complexity algorithm for compile-time task scheduling in heterogeneous systems," *Parallel Computing*, vol. 31, no. 7, pp. 653–670, 2005.
- [9] L. F. Bittencourt, R. Sakellariou, and E. R. Madeira, "Dag scheduling using a lookahead variant of the heterogeneous earliest finish time algorithm," *Proc. Euromicro International Conference on Parallel, Distributed, and Network-Based Processing*, pp. 27–34, 2010.
- [10] R. Sakellariou and H. Zhao, "A hybrid heuristic for dag scheduling on heterogeneous systems," *Proc. IEEE International Parallel and Distributed Processing Symposium*, 2004.
- [11] R. Bajaj and D. P. Agrawal, "Improving scheduling of tasks in a heterogeneous environment," *IEEE Transactions on Parallel and Distributed Systems*, vol. 15, no. 2, pp. 107–118, 2004.
- [12] F. Dong, J. Luo, L. Gao, and L. Ge, "A grid task scheduling algorithm based on qos priority grouping," *Proc. International Conference on Grid and Cooperative Computing*, pp. 58–61, 2006.
- [13] L.-C. Canon, E. Jeannot, R. Sakellariou, and W. Zheng, "Comparative evaluation of the robustness of dag scheduling heuristics," *Grid Computing*, pp. 73–84, 2008.
- [14] S. Pandey, L. Wu, S. M. Guru, and R. Buyya, "A particle swarm optimization-based heuristic for scheduling workflow applications in cloud computing environments," in *Proc. IEEE International Conference on Advanced Information Networking and Applications*, 2010, pp. 400–407.
- [15] T. Ma, S. Pang, W. Zhang, and S. Hao, "A survey of dag scheduling in cloud computing," *Applied Sciences*, vol. 13, no. 7, p. 4580, 2023.